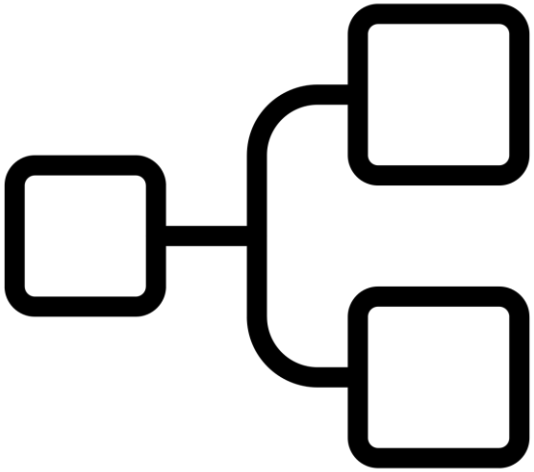


Tipi composti

Struct, enum
25/26

Lezioni 9+ 10
Corso 1

© G. Malnati, G. Liaci, 2021-24

Struct in Rust

- Spesso occorre mantenere unite informazioni tra loro eterogenee
  - Sebbene sia possibile utilizzare una tupla, questa tende a nascondere la semantica complessiva del dato: una tupla contenente due numeri interi potrebbe essere usata per rappresentare una frazione oppure la coordinata di un pixel sullo schermo
  - Quando si intende associare una semantica particolare o legare ad una struttura dati un insieme di comportamenti, è possibile introdurre una **`struct`**
- Una **`struct`** è un costrutto che permette di rappresentare un blocco di memoria in cui sono disposti, consecutivamente, una serie di campi il cui nome e tipo sono indicati dal programmatore

```
struct Player {
    name: String,  // nickname
    health: i32,   // stato di salute (in punti vita)
    level: u8,     // livello corrente
}
```


© G. Malnati, G. Liaci, 2021-24

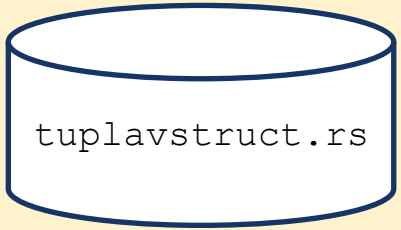
Struct

- Per convenzione, il nome della struct comincia con la lettera maiuscola e utilizza la convenzione CamelCase
  - Per i campi, come per le variabili, le funzioni ed i metodi, si usa la convenzione snake_case
- Si istanzia una struct tramite un blocco preceduto dal nome della struttura, contenente un valore per ciascun campo
  - `let mut s = Player { name: “Mario”.to_string(), health: 25, level: 1 };`
- Si accede ai singoli campi con la notazione puntata (***var.field***)
  - `println!(“Player {} has health {}”, s.name, s.health );`
  - `s.level += 1; //l’accesso in scrittura richiede che s sia mutabile`
- Ogni struct introduce un nuovo tipo, il cui nome coincide con il nome della struct, basato sui tipi che la compongono
  - Questo permette di disambiguare l’uso che si intende fare delle singole informazioni contenute

Struct vs. Tupla

- In una tupla devo mettere i dati nella tupla nell'ordine giusto
- In una struct posso mettere i dati nell'ordine che si preferisce purché siano presenti tutti

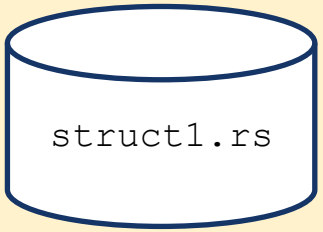
```
#[derive(Debug)]
struct Player {
    name: String,
    health: i32,
    level: u8,}

fn main() {
    let p1 = Player {name: "Mario".to_string(), level: 1, health: 127};

    let t1: (i32, f64, &str) = (42, 3.14, "ciao mamma");

    println!("{:?} {:?}", p1, t1);
}
```



Istanza di una struct

- Si può istanziare una nuova struct a partire da un'altra dello stesso tipo, i campi omessi ricavano i loro valori dalla struct ricevuta
  - `let s1 = Player {name: "Paolo".to_string(), .. s}`

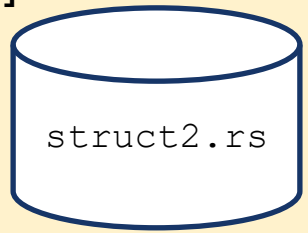
```
#[derive(Debug)]
struct Player {
    name: String,
    health: i32,
    level: u8,
}

fn main() {
    let p1 = Player {name: "Mario".to_string(), health: 25, level: 1} ;
    let p2 = Player {name: "Giovanni".to_string(), .. p1};

    println!("{:?} \n{:?}", p1, p2);
}
```



Istanza con valori di default

- Se i campi implementano il tratto Default è possibile utilizzare il valore di default.

```
#[derive(Debug, Default)]
struct Player {
    name: String,
    health: i32,
    level: u8
}

fn main() {
    let p1 = Player::default();
    let p2 = Player {name: "Mario".to_string(), .. Player::default()};
    println!("{:?}\n{:?}", p1, p2);
}
```

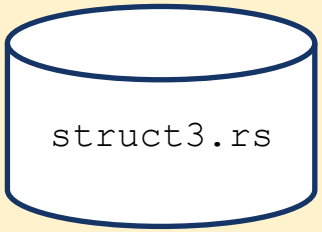



```
Player { name: "", health: 0, level: 0 }
Player { name: "Mario", health: 0, level: 0 }
```

Istanza di una struct

- Quando il nome del valore coincide con quello del campo è possibile abbreviare la notazione

```
#[derive(Debug)]
struct Player {
    name: String,
    health: i32,
    level: u8,
}

fn main() {
    let name = "Mario".to_string();
    let health = 25;
    let level = 1;
    let p1 = Player {name, health, level} ;
    println!("{:?}", p1);
}
```

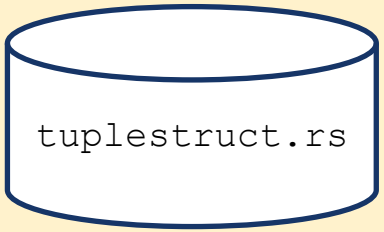


Struct particolari

- Tuple struct:
  - Si possono definire delle struct simili a delle tuple, indicando solo il tipo del campo, senza attribuire un nome
  - Le struct di questo tipo si istanziano come una tupla con l'aggiunta del nome della struct
- Unit struct:
  - si può definire una struct vuota, senza alcun campo, senza allocare memoria, analogamente al tipo **()**
  - Permettono di definire dei comportamenti (tratti) su un tipo che non ha dati.

```
struct Playground ( String, u32, u32 );
struct Empty;   // non viene allocata memoria per questo tipo di valore
                // zero byte type

let mut f = Playground( "football".to_string(), 90, 45 );
let e = Empty;
```

```rust
#[derive(Debug)]
struct Point (i32, i32, i32);

fn main() {

    let p1 = Point(0, 0, 0);  // Tupla con nome di struct
    let p2 = (0, 0, 0);       // Tupla senza nome

    println!("{:?}\n{:?}", p1, p2);
    println!("{} {} {}", p1.0, p1.1, p1.2);
    println!("{} {} {}", p2.0, p2.1, p2.2);
}
```



```rust
#[derive(Copy, Clone, Debug)]
struct Empty;

fn main() {
    let p = Empty;

    let mut array = Vec::new();

    for _ in 0..10
    {
        array.push(p);
    }

    println!("{}", array.len());
    println!("{}", array.capacity());
    println!("{:p}", array.as_ptr());

    for i in 0..10
    {
        println!("{:?}", array[i]);
    }
}
```

```rust
struct Metric;
struct Imperial;

// Calcola la velocità in km/h dato una distanza in chilometri
fn calculate_speed(distance: f64, _unit: Metric) -> f64 {
    distance / 1.0 // distanza per 1 ora, quindi è già km/h
}

// Estensione: calcola la velocità in mph (miglia orarie)
fn calculate_speed_imperial(distance: f64, _unit: Imperial) -> f64 {
    let km_to_miles = 0.621371;
    (distance * km_to_miles) / 1.0
}

fn main() {
    let m = Metric;
    let i = Imperial;
    let distance_km = 120.0;

    let speed_metric = calculate_speed(distance_km, m);
    println!("Velocità in km/h: {:.2}", speed_metric);

    let speed_imperial = calculate_speed_imperial(distance_km, i);
    println!("Velocità in mph: {:.2}", speed_imperial);
}
```

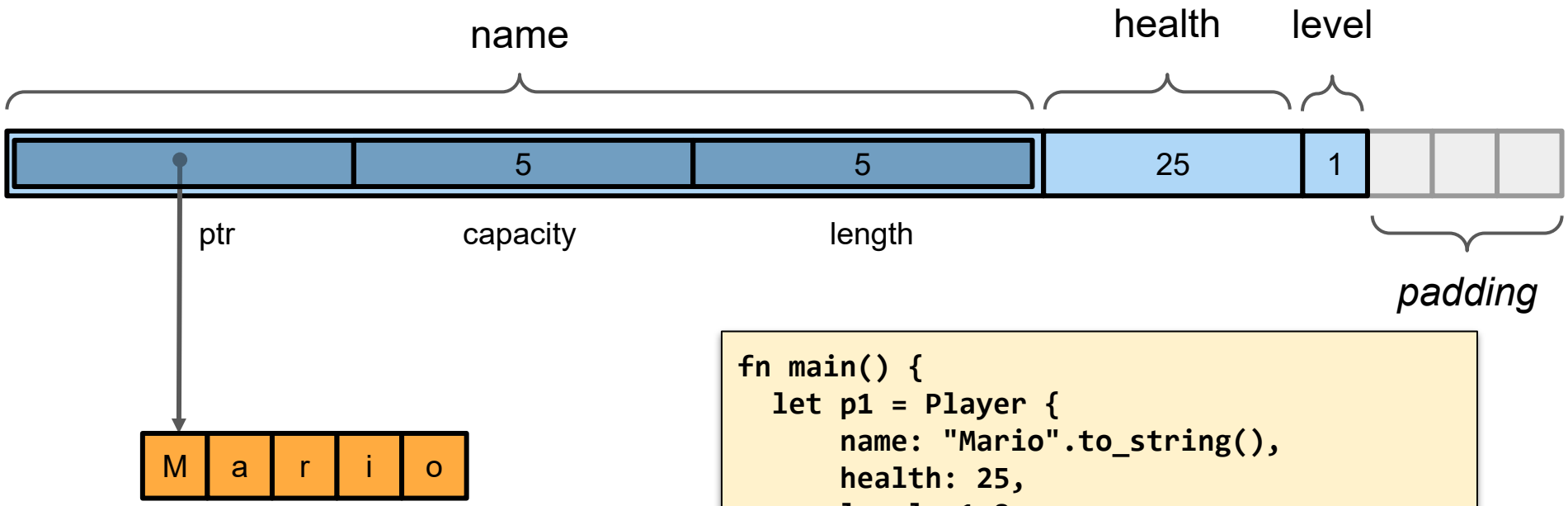
Destrutturazione di una struct

- La destrutturazione è utile per ottenere un “parsing” di una struttura, così da gestirne più facilmente i campi, estraendoli in contenitori singoli da trattare separatamente.
- I campi della struttura generano delle variabili.

```
struct Point {
    x: f32,
    y: f32
}

fn main() {
    let p = Point { x: 5., y: 10. };

    let Point { x: ascissa, y: ordinata} = p;

    println!("The original point was: ({},{})", ascissa, ordinata);
}
```

© G. Malnati, G. Liaci, 2021-24

Probabile rappresentazione in memoria




```
fn main() {
  let p1 = Player {
      name: "Mario".to_string(),
      health: 25,
      level: 1u8,
  };
  println!("{:?}", p1);
}
```

Rappresentazione in memoria

- La disposizione in memoria dei singoli campi è scelta arbitrariamente da Rust ed è conseguenza di vincoli ed ottimizzazioni e può essere controllata attraverso opportuni meccanismi
  - Ogni singolo campo, in base al proprio tipo, richiede che l'indirizzo a cui viene collocato sia multiplo di una data potenza di 2 (allineamento)
    - La funzione `std::mem::align_of_val(...)` permette di conoscere l'allineamento richiesto da un particolare valore, mentre la funzione `std::mem::size_of_val(...)` ne indica la dimensione
  - I vincoli di allineamento dipendono dalla piattaforma di esecuzione (modello di CPU)
- L'allineamento e la disposizione di una struct è controllata attraverso l'attributo `#[repr(...) ]` anteposto alla dichiarazione della struct
  - In assenza di tale attributo, viene assunta la rappresentazione di default, che lascia libero il compilatore di riordinare la sequenza dei campi, per ottimizzare l'accesso
  - Indicando `#[ repr(C) ]`, si ottiene una rappresentazione coerente con le regole di interfaccia binaria definite dal linguaggio C, fondamentali per l'interoperabilità con librerie scritte in altri linguaggi

Piccolo ripasso

- Quanto è grande un riferimento ad un intero?
- Quanto è grande un riferimento ad un array?
- Quanto è grande un riferimento ad un Vec?
- Quanto è grande un riferimento ad un Box di i32?
- Quanto è grande un riferimento ad un Box di una tupla di i32 e bool?
- Quanto è grande un riferimento ad un Box di uno slice?
- Quanto è grande un Box di uno slice ?

© G. Malnati, G. Liaci, 2021-24

size.rs

```rust
fn main() {
    use std::mem::size_of;
    // Su sistema a 64-bit, un puntatore = 8 byte
    // 1. Riferimento a un intero → thin pointer
    println!("&i32            = {} byte", size_of::<&i32>());
    // 2. Riferimento a un array → thin pointer (dimensione nota a compile-time)
    println!("&[i32; 10]      = {} byte", size_of::<&[i32; 10]>());
    // 3. Riferimento a un Vec → thin pointer (Vec è Sized)
    println!("&Vec<i32>       = {} byte", size_of::<&Vec<i32>>());
    // 4. Riferimento a Box<i32> → thin pointer (Box<i32> è Sized)
    println!("&Box<i32>       = {} byte", size_of::<&Box<i32>>());
    // 5. Riferimento a Box<(i32, bool)> → thin pointer
    println!("&Box<(i32,bool)>= {} byte", size_of::<&Box<(i32, bool)>>());
    // 6. Riferimento a Box<[i32]> → thin pointer verso un fat pointer
    println!("&Box<[i32]>     = {} byte", size_of::<&Box<[i32]>>());
    // 7. Box<[i32]> è un fat pointer (ptr + len)
    println!("Box<[i32]>  = {} byte", size_of::<Box<[i32]>>());
}
```

Text

lezione del 16-03-2026 —> ho sovrascritto il file e l'ho persa, se serve riascoltala online

Visibilità

- I **moduli** permettono di implementare il meccanismo di **incapsulamento** *(information hiding)*

- In Rust ci sono 2 livelli di visibilità:
  - Una struct è visibile solo nel modulo in cui è dichiarata (default) o nei sottomoduli
  - Aggiungendo il modificatore di visibilità (**pub**) si permette di far vedere la struct anche al di fuori del modulo (o del sotto-modulo) in cui è dichiarata.

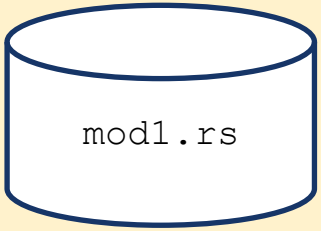
- Sia la struct nel suo complesso che i singoli campi che la formano possono essere preceduti dal **modificatore di visibilità**
  - Tale modificatore ha impatto sull'accesso al contenuto della struct da parte di codice presente in moduli diversi da quello in cui la struct è stata definita
  - Di base, **i campi sono** considerati **privati** (accessibili solo al codice del modulo corrente e ai suoi sotto-moduli): possono però essere resi pubblici facendo precedere il nome dalla parola chiave **pub**

© G. Malnati, G. Liaci, 2021-24

Moduli

- E' possibile organizzare il codice su più moduli indipendenti

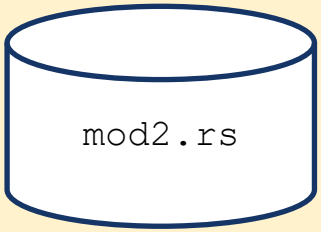
```rust
mod Module1 {

struct Test{
          a: i32,
          b: bool
          }
}

fn main() {
    let t = Test { a: 12, b: false};
    println!("Hello, world!");
}
```




```
11 |     let t = Test { a: 12, b: false};
   |             ^^^^ not found in this scope
   |
note: struct `crate::Module1::Test` exists but is inaccessible
  --> src\main.rs:3:1
   |
3  | struct Test{
   | ^^^^^^^^^^^ not accessible
```

Moduli

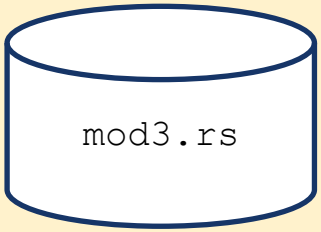
```
mod Module1 {

struct Test{
    a: i32,
    b: bool
           }
}

use Module1::Test;

fn main() {
    let t = Module1::Test { a: 12, b: false};
    println!("Hello, world!");
}
```




```
15 | use Module1::Test;
   |             ^^^^ private struct
   |
note: the struct `Test` is defined here
  --> src\main.rs:3:1
   |
3  | struct Test{
   | ^^^^^^^^^^^
```

Moduli

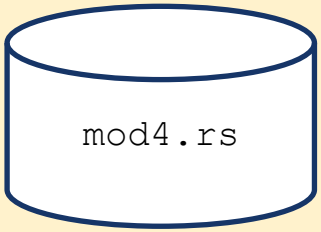
```
mod Module1 {

pub struct Test{
    a: i32,
    b: bool
  }

}

use Module1::Test;

fn main() {
    let t = Module1::Test { a: 12, b: false};
    println!("Hello, world!");
}
```




```
18 |     let t = Module1::Test { a: 12, b: false};
   |                            ^^^^^ private field
```

Moduli

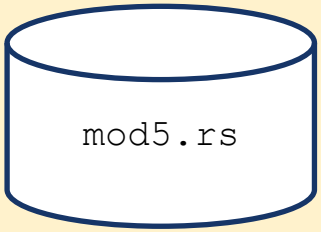
```
mod Module1 {

pub struct Test{
    pub a: i32,
    pub b: bool
  }
}

use Module1::Test;
fn main() {
    let t = Test { a: 12, b: false};
    println!("Hello, world!");
}
```



Incapsulamento

Permette di **nascondere i dettagli implementativi interni** e esporre un'interfaccia pubblica stabile, promuovendo la modularità e prevenendo modifiche esterne indesiderate.

```
mod Module1 {

#[derive(Debug)]
pub struct Test{
     a: i32,
     b: bool
}
pub fn f (i: i32, boo: bool) -> Test
{
    Test {a: i, b: boo}
}

}
use Module1::Test;
use Module1::f;
fn main() {
    let t: Test = f(100, true);
    println!("{:?}", t);
}
```



Benefici dell'Incapsulamento in Rust

- Controllo dell'Accesso: Permette di definire un'interfaccia pubblica stabile per un modulo o un tipo, nascondendo i dettagli implementativi interni che potrebbero cambiare.
- Prevenzione di modifiche involontarie: Impedisce al codice esterno di manipolare direttamente lo stato interno di una struttura in modi non previsti, mantenendo l'integrità dei dati.
- Modularità e organizzazione: Aiuta a organizzare il codice in unità logiche e a ridurre l'accoppiamento tra diverse parti del programma.
- Manutenibilità: Rendendo esplicita l'interfaccia pubblica, è possibile modificare l'implementazione interna senza rompere il codice che utilizza l'interfaccia.

© G. Malnati, G. Liaci, 2021-24

Metodi

- Per essere efficace, occorre però poter associare un insieme di **comportamenti** (**metodi**) alla struct
- A differenza di quanto avviene in altri linguaggi, in cui struttura e comportamento sono definiti contestualmente in un unico blocco (classe), in Rust la definizione dei metodi associati ad una `struct` avviene **separatamente**, in un blocco di tipo `impl` …

© G. Malnati, G. Liaci, 2021-24

Metodi

Altri linguaggi

(C++, Java, Javascript ES6+, …)

```
class Something {

  int i;                          Dati
  String s;

  void process() {...}            Metodi
  int increment() {...}

};
```

Rust

```
struct Something {
  i: i32,                         Dati
  s: String
}

impl Something {

  fn process(&self) {...}         Metodi
  fn increment(&mut self) {...}

}
```

Metodi

- Rust non è un linguaggio ad oggetti nel senso tradizionale del termine e, conseguentemente, **non ha** il concetto di classe
  - Sebbene le struct possano apparire simili alle classi di altri linguaggi, il parallelismo è limitato
  - In particolare, le struct **NON** sono organizzate in una gerarchia di **ereditarietà**
    - Una struct non può estendere un'altra struct, ma ogni struct è indipendente
  - Il concetto di metodo si applica invece a tutti i tipi, compresi quelli primitivi
- Si definiscono i metodi collegati ad un tipo in un blocco racchiuso tra parentesi graffe, preceduto dalla parola chiave `impl` seguita dal nome del tipo
  - Le funzioni presenti in tale blocco il cui primo parametro sia `self` (una parola chiave che rappresenta l'istanza del tipo di cui si sta facendo l'implementazione), `&self` o `&mut self` diventano **metodi** (*self* corrisponde grosso modo a quello che in altri linguaggi ad oggetti viene chiamato *this*)

© G. Malnati, G. Liaci, 2021-24

Metodi

- I metodi sono funzioni legate ad un'istanza di un dato tipo (sia a livello sintattico, che a livello semantico)
- Sintatticamente, **un metodo viene invocato a partire da un'istanza** del tipo a cui è legato
  - Si usa la notazione *instance.method(…)*, dove *instance* è una variabile del tipo dato (detto anche **ricevitore** del metodo) e *method* è il nome della funzione
- Semanticamente, il codice del metodo ha accesso al contenuto (pubblico e privato) del ricevitore attraverso la parola chiave `self`
  - Posso specificare in modi diversi come il metodo tratta la struttura che prende in ingresso (il ricevitore del metodo)
  - Di fatto, i metodi legati ad una struct vengono implementati sotto forma di funzioni con un parametro ulteriore (chiamato `self`, `&self` o `&mut self`) il cui tipo è vincolato alla struct per la quale sono definiti

© G. Malnati, G. Liaci, 2021-24

Metodi

- Il primo parametro di un metodo definisce il livello di accesso che il codice del metodo ha sul ricevitore
  - `self` indica che il ricevitore viene passato **per movimento**, di fatto consumando il contenuto della variabile: è una forma contratta della notazione `self: Self` (il tipo Self coincide con il tipo della struct)
  - `&self` indica che il ricevitore viene passato **per riferimento condiviso**: è una forma contratta di `self: &Self`
  - `&mut self` indica che il ricevitore viene passato **per riferimento esclusivo**: è una forma contratta di `self: &mut Self`
- `Self` (parola chiave) rappresenta il tipo che sto implementando
- Se presente, il parametro self compare come **primo elemento** nella dichiarazione del metodo
  - All'atto dell'invocazione del metodo, esso è ricavato implicitamente dal valore che compare a sinistra del punto che precede il nome del metodo
- Le funzioni che non hanno come primo parametro `self` sono dette **funzioni associate** e svolgono il ruolo giocato dai costruttori e dai metodi statici nei linguaggi ad oggetti

Metodi

```
struct Point {
  x: i32,
  y: i32,
}

impl Point {

  fn mirror(self) -> Self {
    Self{ x: self.y, y: self.x }
  }



 fn length(&self) -> f64 {
        let a = self.x*self.x + self.y*self.y;
        let res = (a as f64).sqrt();
        res
    }

fn scale(&mut self, s: i32) {
    self.x *= s;
    self.y *= s;
  }
}
```

In tutto il blocco, Self coincide con Point

In RUST occorre specificare il nome completo self.field

Consuma una struct Point e **produce** una nuova struct dello stesso tipo

CONSUMO

Opera su una struct Point senza possederla né mutarla

LEGGO

Opera su una struct Point cambiandone il contenuto

MODIFICO

Metodi

```
fn main() {


  let p1 = Point{ x: 3, y: 4 };
  let mut p2 = p1.mirror();

  let l1 = p2.length(); // l1: 5

  p2.scale(2);

  let l2 = p2.length();
  // l2: 10
}
```

point.rs

`p1` non potrà più essere usato dopo questa linea: il suo valore è stato mosso nel parametro `self` del metodo `mirror()`

Al parametro `self` del metodo `length()` è stato legato un riferimento condiviso a `p2`: tale riferimento cessa di esistere quando il metodo ritorna

Al parametro `self` del metodo `scale(...)` è stato legato un riferimento mutabile a `p2`: tale riferimento cessa di esistere quando il metodo ritorna

Parametri aggiuntivi

```
struct Rettangolo {
    base: u32,
    altezza: u32,
}
impl Rettangolo {
    fn contiene (&self, altro: &Self) -> bool {
        self.base > altro.base && self.altezza > altro.altezza
    }
}
fn main() {
    let rett1 = Rettangolo {base: 30, altezza: 50};
    let rett2 = Rettangolo {base: 10, altezza: 40};
    let rett3 = Rettangolo {base: 60, altezza: 45};

    println!("Il Rettangolo 1 contiene il Rettangolo 2? {}", rett1.contiene(&rett2));
    println!("Il Rettangolo 1 contiene il Rettangolo 3? {}", rett1.contiene(&rett3));
}
```

```
Il Rettangolo 1 contiene il Rettangolo 2? true
Il Rettangolo 1 contiene il Rettangolo 3? false
```

Costruttori

- In C++, tutte le classi contengono metodi particolari detti **costruttori**
  - Hanno il compito di inizializzare le istanze della classe
  - Se non vengono scritti esplicitamente dal programmatore, il compilatore provvede a generarne alcuni (costruttore di default, privo di parametri, costruttore di copia, con un solo parametro di tipo riferimento costante ad un'istanza della classe corrente)
- In Rust, non esiste il concetto di costruttore
  - Qualunque frammento di codice, in un qualunque modulo che abbia visibilità di una data struct e dei suoi campi, può crearne un'istanza, indicando un valore per ciascun campo
  - Questo garantisce che il programmatore sia consapevole delle informazioni contenute al suo interno
- Per evitare significative duplicazioni di codice e favorire l'incapsulamento, le implementazioni spesso includono metodi statici per l'inizializzazione delle istanze
  - Per **convenzione**, un metodo di questo tipo viene chiamato `pub fn new() -> Self {...}`
  - Poiché Rust **non supporta l'overloading delle funzioni**, se servono più funzioni di inizializzazione, ciascuna di esse avrà un nome differente: in questo caso la convenzione è utilizzare un pattern come `pub fn with_details(...) -> Self {...}` (per convenzioni queste funzioni cominciano con with).

Costruttore

- Un metodo costruttore è un metodo senza ricevitore.

```
#[derive(Debug)]
pub struct Test{
        a: i32,
        b: bool
    }
    impl Test {
        fn new() -> Test {
            Test {a: 0, b: false}
        }
    fn with_initial_values (i: i32, boo: bool) -> Test {
        Test {a: i, b: boo}
    }
    fn evaluate(&self) -> bool {
        if self.b == false {false}
        else if self.a != 0 {true} else {false}
     }
    }
fn main() {
    let mut t =Test::new();
    println!("{:?} {:?}", t, t.evaluate());
    t = Test::with_initial_values(100, true);
    println!("{:?} {:?}", t, t.evaluate());

}
```

Costruttore (II)

```
#[derive(Debug)]
struct Player {
    name: String,
    health: i32,
    level: u8,
}

impl Player {
    fn with_name(s: String) -> Self {
        Self {name: s, health: 25, level: 1}
        }
    }
fn main() {
    let p1 = Player::with_name("Mario".to_string());

    println!("{:?}", p1);
}
```

Distruttori

- In C++, ogni classe prevede un particolare metodo detto **distruttore**
  - Il suo compito è rilasciare le risorse possedute dall'istanza della classe
  - Ha una sintassi particolare: il suo nome coincide con il nome della classe preceduto dal segno **~** (tilde)
  - Il compilatore chiama automaticamente questo metodo se l'oggetto esce dallo scope sintattico (al termine cioè del suo naturale ciclo di vita) o se viene rilasciato esplicitamente (in quanto ospitato sullo heap e distrutto tramite l'operatore **`delete`**)
  - Se il programmatore non definisce questo metodo, il compilatore provvede a generare un'implementazione vuota
- La presenza del distruttore abilita, in C++, un particolare approccio detto ***Resource Acquisition Is Initialization*** (RAII)
  - Poiché il distruttore è chiamato automaticamente quando una variabile locale esce dallo scope, si possono usare costruttore e distruttore in coppia per garantire che determinate azioni siano eseguite in un blocco di codice in cui sia presente una variabile locale appositamente dichiarata

Il nome giusto sarebbe dovuto essere
Resource Finalization is Destruction (RFID)

Distruttori

- Rust gestisce il rilascio di risorse contenute in un'istanza attraverso il tratto **Drop**
  - Tale tratto è costituito dalla sola funzione **`drop(&mut self) -> ()`**
  - Il compilatore riconosce la presenza di questo tratto nei tipi definiti dall'utente e provvederà a chiamare la funzione che lo costituisce quando le variabili di quel tipo escono dal proprio scope sintattico
  - Si può forzare il rilascio delle risorse contenute in un oggetto usando la funzione **`drop(some_object);`** che ne acquisisce il contenuto e determina l'uscita dallo scope

```rust
pub struct Shape {

  pub position: (f64, f64),
  pub size: (f64, f64),
  pub type: String

}
```

```rust
impl Drop for Shape {

    fn drop(&mut self) {
        println!("Dropping shape!");
    }
}
```

Distruttori

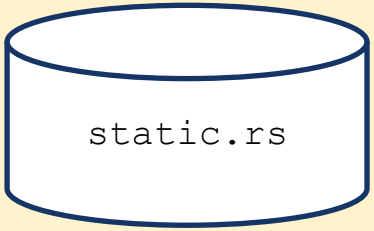
- Il paradigma RAII viene mutuato dal C++ e costituisce un importantissimo modo per gestire automaticamente acquisizione e rilascio di risorse
  - Oltreché permettere l'esecuzione automatica di coppie di funzioni
  - Può essere usato ogni qual volta sia necessario garantire il corretto rilascio di risorse di sistema, come la memoria allocata sullo heap, handle di file, socket, ...
- Per poter implementare correttamente il concetto di RAII bisogna essere sicuri che, per ogni costruzione, ci sia una sola invocazione Drop e per questo motivo ….
- Il tratto **Drop** è **mutuamente esclusivo** con il tratto **Copy**
  - Se un tipo implementa il primo non può implementare l'altro, e viceversa

Metodi statici

- In **C++** è lecito inserire all'interno del costrutto `class … { }` la dichiarazione di metodi preceduti dalla parola chiave `static`
  - Essi non sono legati ad una specifica istanza, ma possono operare sulle istanze della classe (se ne conoscono l'indirizzo) avendo accesso anche alle componenti private
- In **Rust**, è possibile implementare metodi analoghi semplicemente **non indicando**, come primo parametro, né `self` né un suo derivato
- Sono anche chiamate **associated functions** e sono definite su un tipo anziché su un'istanza di un tipo
  - permettono la creazione di funzioni per la costruzione di un'istanza, metodi per la conversione di istanze di altri tipi nel tipo corrente o, semplicemente, l'accesso a funzionalità statiche (come nel caso di librerie matematiche o l'accesso in lettura di parametri di configurazione)
  - la chiamata in questo caso sarà usando `<Tipo>::metodo()`
  - Esempi:
    - `let s = String::from("Hello world!");`
    - `let mut arr = Vec::new();`
  -

Metodi Statici

```rust
struct Rectangle {
    width: u32,
    height: u32,
}
impl Rectangle {
    fn new(width: u32, height: u32) -> Rectangle {
        Rectangle { width, height }
    }
    fn with_square(size: u32) -> Rectangle {
        Rectangle { width: size, height: size }
    }
    fn calculate_area(&self) -> u32 {
        self.width * self.height
    }
}
fn main() {
    let rectangle = Rectangle::new(5, 10);
    let area = rectangle.calculate_area();
    println!("Area: {}", area);
    let square = Rectangle::with_square(10);
    let area = square.calculate_area();
    println!("Area: {}", area);
}
```



Enum

- In Rust, è possibile introdurre tipi enumerativi composti da un semplice valore scalare (come in C e C++)
  - Ma anche incapsulare più **tuple** o più **struct** (che si pongono in alternativa una rispetto ad un'altra) volte a fornire ulteriori informazioni relative allo specifico valore
- Inoltre, è possibile legare **metodi** ad un'enumerazione
  - Aggiungendo un blocco impl … come nel caso delle struct

```
enum HttpResponse {
  Ok = 200,
  NotFound = 404,
  InternalError = 500
}
```

```
enum HttpResponse {
  Ok,
  NotFound(String),
  InternalError {
    desc: String,
    data: Vec<u8> },
}
```

Enum

- Enum è definito come tipo **somma**
  - L'insieme dei valori che può contenere è l'unione dei valori delle singole alternative
- Struct è un tipo **prodotto**: l'insieme dei valori che può contenere è il prodotto cartesiano degli insiemi legati ai singoli campi

- La possibilità di legare, agli specifici valori, uno o più dati è alla base di molti pattern di programmazione tipici di Rust (ad es. la gestione dell'opzionalità e la rappresentazione del risultato di una computazione che può fallire)

© G. Malnati, G. Liaci, 2021-24

Rappresentazione in memoria

- La rappresentazione in memoria di un enum in Rust dipende dalla natura delle varianti e dalla presenza o meno di dati associati a esse.

© G. Malnati, G. Liaci, 2021-24

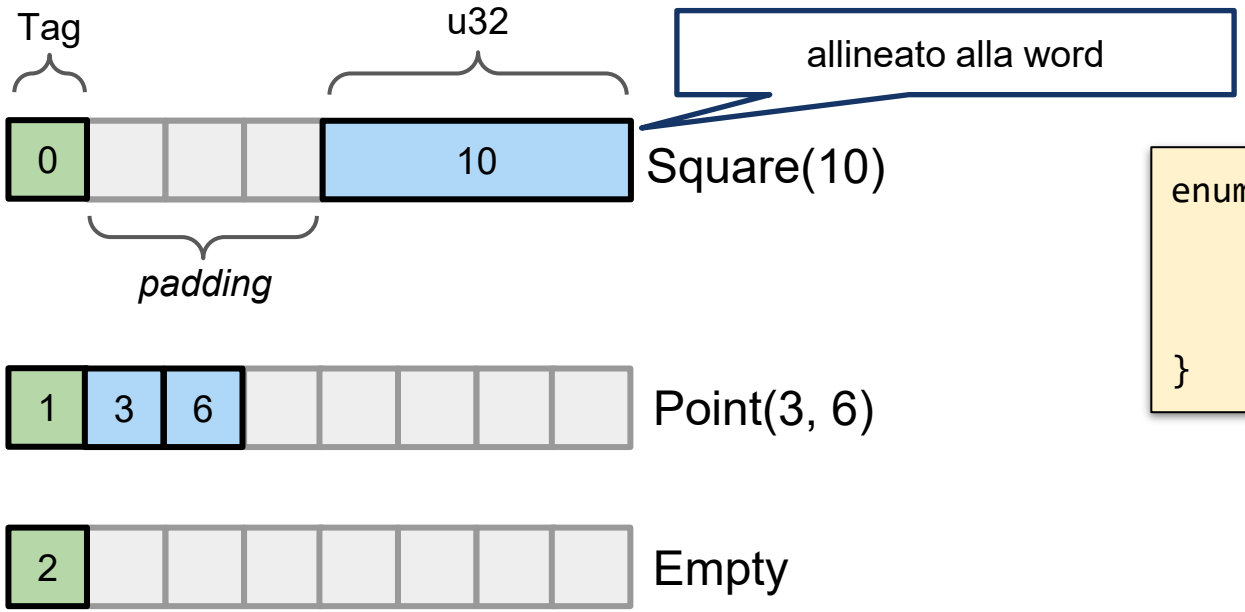
Enum senza dati

```rust
enum Color {
  Red,
  Green,
  Blue,
}
```

- Rust utilizza un'implementazione basata su "tag", dove ogni variante è rappresentata da un numero intero univoco. In questo caso, la memoria necessaria per rappresentare un'istanza di `Color` è equivalente a quella necessaria per archiviare un intero.

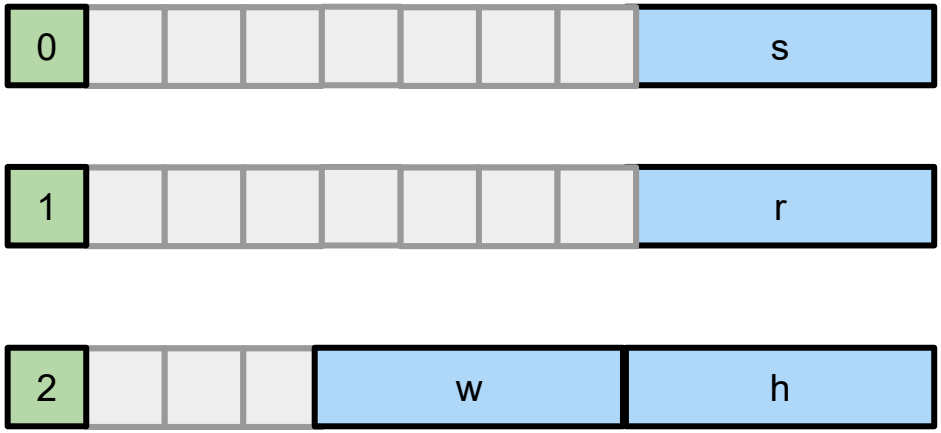
Rappresentazione in memoria

- Rust deve allocare memoria sia per il selettore (tag) che identifica la variante, sia per i dati associati alla variante
- In memoria gli enum occupano lo spazio di un intero da 1 byte più lo spazio necessario a contenere la variante più grande




```
enum Shape {
        Square(u32),
        Point(u8, u8),
        Empty,
}
```

Rappresentazione in memoria (II)

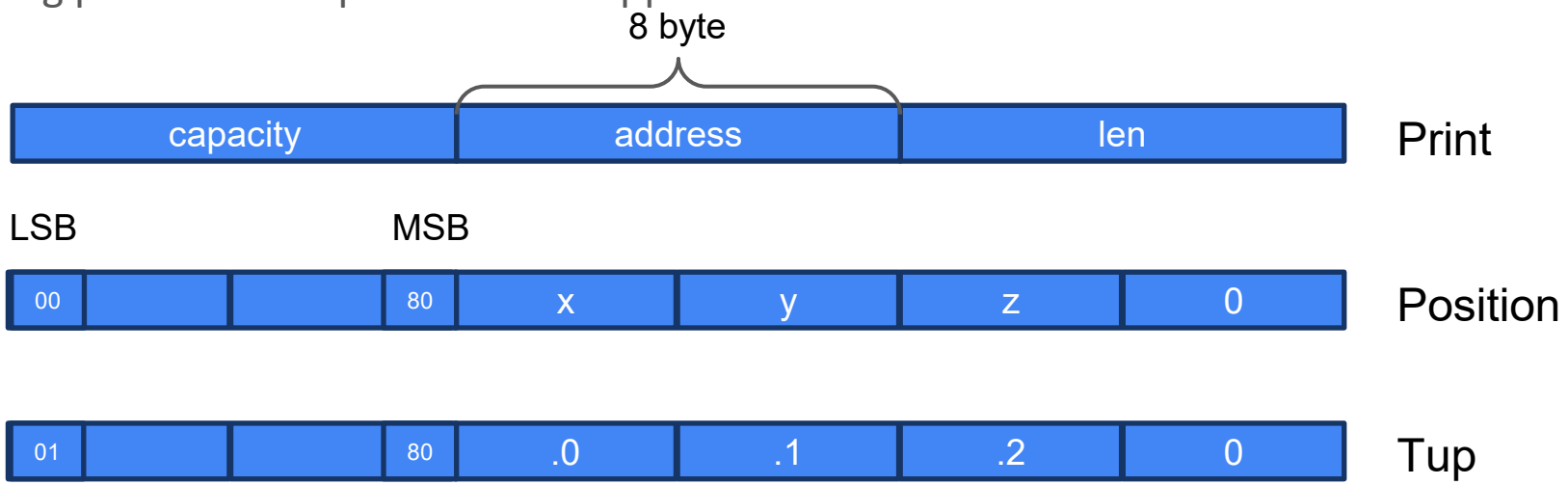



```
enum Shape {
    Square { s: i32 },
    Circle { r: i32 },
    Rectangle { w: i32, h: i32 }
}
```

Ottimizzazione

```
enum MyEnum {
    Print(String),
    Position { x:i32, y:i32 , z:i32},
    Tup(i32, i32, i32),
}
```

- La rappresentazione in memoria è progettata per essere efficiente e compatta e il tag può essere implicito e non rappresentato

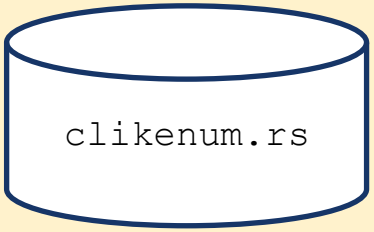


- La *capacity* di una stringa NON può essere negativa
- Il primo byte delle altre varianti è usato per rappresentare il tag.

© G. Malnati, G. Liaci, 2021-24

C-like Enum

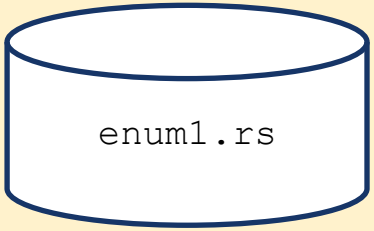
```
#![allow(dead_code)]

#[derive(Debug)]
enum Color {
    Red,
    Green = 50,
    Yellow
}

fn main() {
    println!("Colore Rosso {:?} {:?}", Color::Red, Color::Red as i32);
    println!("Colore Giallo {:?} {:?}", Color::Yellow, Color::Yellow as i32);
}
```




```
Colore Rosso Red 0
Colore Giallo Yellow 51
```

Enum

```rust
#![allow(dead_code)]

fn main() {
    // define enum with multiple variants and data types
    #[derive(Debug)]
    enum Game {
        Quit,
        Print(String),
        Position { x: i32, y: i32 },
        ChangeBackground(i32, i32, i32),
    }

    // initialize enum with values
    let quit = Game::Quit;
    let print = Game::Print(String::from("Hello World!"));
    let position = Game::Position { x: 10, y: 20 };
    let color = Game::ChangeBackground(200, 255, 255);

    // print enum values
    println!("quit = {:?}", quit);
    println!("print = {:?}", print);
    println!("position = {:?}", position);
    println!("color = {:?}", color);
}
```



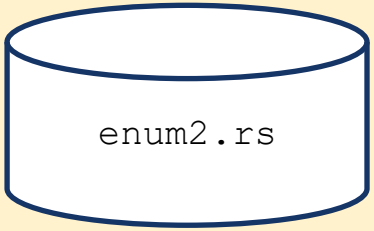
Enumerazioni e clausole match

- Il costrutto **`match…`** si presta particolarmente per gestire in modo differenziato valori enumerativi
  - Il comportamento offerto dal pattern matching e la possibilità di legare variabili temporanee in base alla struttura del valore che viene analizzato offre una sintassi compatta ed efficiente per esprimere comportamenti alternativi (**destructuring assignment**)
  - Il fatto che i pattern confrontati debbano essere esaustivi, garantisce che il codice resti coerente anche se il numero di possibili alternative presenti nell'enumerazione cambia nel tempo

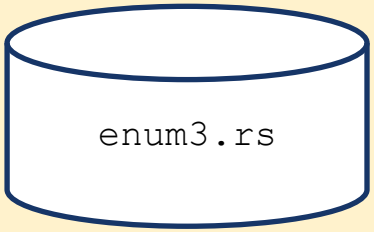
```
enum Shape {
    Square { s: f64 },
    Circle { r: f64 },
    Rectangle { w: f64, h: f64 }
}
```

```
fn compute_area(shape: Shape) -> f64 {
    match shape {
        Shape::Square { s } => s*s,
        Shape::Circle { r } => r*r*3.1415926,
        Shape::Rectangle {w, h} => w*h,
    }
}
```

```rust
enum Shape {
    Square { s: f64 },
    Circle { r: f64 },
    Rectangle { w: f64, h: f64 }
}

fn compute_area(shape: Shape) -> f64 {
    match shape {
        Shape::Square { s } => s*s,
        Shape::Circle  { r } => r*r*3.1415926,
        Shape::Rectangle {w, h} => w*h,
    }
}

fn main() {
    // initialize enum with values
    let quadrato = Shape::Square{s: 1.0};
    let cerchio = Shape::Circle{r: 2.0};
    let rettangolo = Shape::Rectangle{w: 2.0, h: 3.0};

    println!("Area del Quadrato {}", compute_area(quadrato));
    println!("Area del Cerchio {}", compute_area(cerchio));
    println!("Area del Rettangolo {}", compute_area(rettangolo));
}
```



```rust
enum Operazioni {
    Add,
    Subtract,
}
impl Operazioni {
    fn run(&self, x: i32, y: i32) -> i32 {
        match self {
            Self::Add => x + y,
            Self::Subtract => x - y,
        }
    }
}
fn main() {
let op1 = Operazioni::Add;
let op2 = Operazioni::Subtract;

    let res = op1.run(5,7);
    println!("{}", res);
    let res = op2.run(5,7);
    println!("{}", res);
}
```




```
12
-2
```

Destrutturazione

- L'utilizzo del pattern matching e la possibilità di destrutturare un valore complesso non sono limitate al costrutto `match …`

- Quando non è necessario esplorare tutti i casi possibili si possono usare i costrutti
  `if let <pattern> = <value> …`
  `while let <pattern> = <value>`
  - Tali costrutti verificano se il valore fornito corrisponda o meno al pattern indicato e, nel caso, eseguono le necessarie assegnazioni alle variabili contenute nel pattern

```
fn process(shape: Shape) {
        // stampa solo se shape è Square…
    if let Shape::Square { s } = shape {
            println!("Square side {}", s);
    }
}
```

```rust
#![allow(dead_code, unused_variables)]

enum Shape {
    Square { s: f64 },
    Circle { r: f64 },
    Rectangle { w: f64, h: f64 }
}

fn process (shape: Shape) -> () {
    if let Shape::Square { .. } = shape {
        println!("E' un quadrato")
    }
    else {
        println!("Non è un quadrato")
    }
}

fn main() {
    let cerchio = Shape::Circle{r: 2.0};
    process(cerchio);
}
```

..
per omettere i campi della variante

iflet1.rs

```rust
#![allow(dead_code)]
#[derive(Debug)]
enum Shape {
    Square { s: f64 },
    Circle { r: f64 },
    Rectangle { w: f64, h: f64 }
}

fn process (shape: & mut Shape) {
    if let Shape::Square {s} = shape {
        *s *= 2.0;
    }

}
fn main() {
    let mut square = Shape::Square{s: 4.0};
    process(& mut square);
    println!(" {:?}", square);
}
```

Text

Enumerazioni generiche

lezione del 17-03-2026

enumerazioni: sono di tipo sommo
dichiaro enum selezionando una delle varianti

- Come verrà meglio presentato in seguito, è possibile definire tipi generici
  - Ovvero costrutti che contengono dati il cui tipo è specificato attraverso una meta-variabile, indicata accanto al nome del tipo, racchiusa tra i simboli '<' e '>'
  - I frammenti di codice che utilizzano un tipo generico hanno il compito di indicare quale sia il tipo concreto da sostituire alla meta-variabile
  - Vec<T>, ad esempio, rappresenta un generico vettore di valori omogenei di tipo T

Text

- Rust offre due importanti enumerazioni generiche, che sono alla base della sua libreria standard
  - **`Option<T>`** - rappresenta un valore di tipo T **opzionale** (ovvero che potrebbe non esserci)
  - **`Result<T,E>`** - rappresenta **alternativamente** un valore di tipo T o un errore di tipo E

Text

enum può essere usato moltissimo
per specificare categorie di errori

ci sono due enum disponibili nella libreria standard, che sono molto utili
1) Option: permette di fornire una risposta sulla opzionalità del dato. fa uso di una funzione se che restituisce il dato oppure c'è il caso in cui il dato non esiste
2) Result <T,E>: l'enum result usa due tipi e vedremo in seguito in che modo vengono usati. per ora ci limitiamo a definire la firma dei due enum

T ed E sono chiamati PARAMETRO DI TIPO: “T” è un simbolo che nasconde un tipo generico, anche se occorre sempre specificare che cos’è T.

le parentesi angolari indicano i parametri di tipo che troviamo nella definizione della enum

`Option<T>`

serve per dire se esiste o non esiste un valore

- Tipo di Enum definita nella *standard library* che permette di rappresentare il caso in cui un valore può essere presente o assente

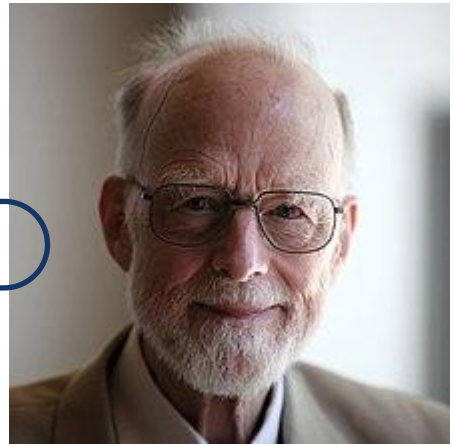
La semantica di none, o la variante None, serve per dire che l’informazione non esiste
La variante Some serve per dire che il valore esiste ed è tipo T ed è incapsulata nella variante Some

- **`Option<T>`** contiene due possibili valori
  - **`Some(T)`** - indica la presenza e contiene il valore
  - **`None`** - indica che il valore è assente

- L’istruzione **`match`** **…** risulta particolarmente utile con questo tipo di valori

le enum servono quindi per fare un analisi delle varianti, e per farlo si possono usare due modi
1) if let —> selettivo, usato quando mi interessa uno specifico tipo
2) match —> esaustivo

Null References: The Billion Dollar Mistake



Lo chiamo il mio errore da un miliardo di dollari.
Fu l'invenzione del riferimento nullo nel 1965. A quel tempo stavo progettando il primo sistema di tipi completo per i riferimenti in un linguaggio orientato agli oggetti.
Il mio obiettivo era garantire che tutti gli usi dei riferimenti fossero assolutamente sicuri, con il controllo eseguito automaticamente dal compilatore. Ma non ho potuto resistere alla tentazione di inserire un **riferimento nullo**, semplicemente perché era così facile da implementare. Ciò ha portato a **innumerevoli errori**, vulnerabilità e arresti anomali del sistema, che probabilmente hanno causato un miliardo di dollari di dolore e danni negli ultimi quarant'anni.

Tony Hoare, (25 August 2009) "Null References: The Billion Dollar Mistake". InfoQ.com
https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/

se vedo una funzione che ritorna un Option significa che ho due possibili varianti, il caso in cui ci sia il valore di ritorno e il caso in cui non ci sia.

Esempio

```
fn divide(numerator: f64, denominator: f64) -> Option<f64> {
    if denominator == 0.0 {
        None
    } else {
        Some(numerator / denominator)
    }
}

fn main() {
    let result = divide(820.0, 15.2);

    match result {
        Some(value) => println!("La divisione è {}.", value),
        None => println!("Non è possibile eseguire la divisione."),
    }
}
```

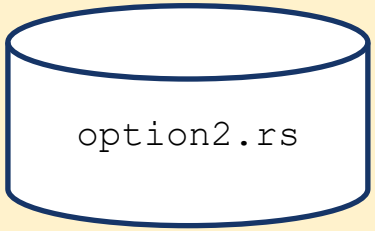
result è di tipo Option
quando la funzione restituisce il risultato

option1.rs

nel caso “Some” significa che acquisisco value e lo printo in questo caso.

l’enum ha il tag, che serve per distinguere le varianti. Anche se None non porta alcun valore, in realtà il compilatore riconosce il tag associato alla variante

```rust
fn divide(numerator: f64, denominator: f64) -> Option<f64> {
    if denominator == 0.0 {
        None
    } else {
        Some(numerator / denominator)
    }
}

fn raddoppia (x: Option<f64> ) -> Option<f64> {
         match x {
                  None => None,
                  Some(i) => Some(i*2.0),
                  }
}

fn main() {
    let result = divide(820.0, 15.2);

    let numero = raddoppia (result);
    match numero {
         Some(numero) => println!("Risultato della divisione moltiplicato per 2 vale {}",
numero),
        None => println!("Non è possibile eseguire la divisione."),
    }
}
```



Enumerazioni generiche

11:50

- `Result<T,E>` si usa per indicare l'esito di un computazione; può valere:
  - `Ok(T)` - Se la computazione ha avuto successo, il valore restituito ha tipo T
  - `Err(E)` - Se la computazione è fallita, il tipo E viene usato per descrivere la ragione del fallimento

le parentesi angolari rappresentano una firma dei tipi che utilizzati. Il tipo T viene usato nella variante Ok; il tipo E viene usato nella variante Err

Esempio

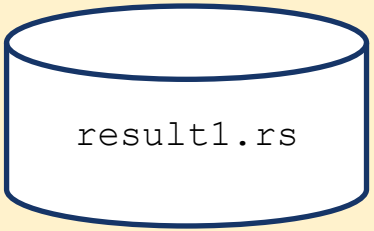
annotazione del tempo di vita, e quindi è una stringa che vive per tutta la durata del programma

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {
    if y == 0.0 {
        // Returning an error if division by zero is attempted
        Err("Division by zero!")
    } else {
        // Returning the result of the division
        Ok(x / y)
    }
}

fn main() {
    // Attempting division
    let dividend = 10.0;
    let divisor = 2.0;

    // Handling the result
    match divide(dividend, divisor) {
        Ok(result) => println!("Result: {}", result),
        Err(err) => println!("Error: {}", err),
    }
}
```

il compilatore leggendo il tag capisce quale variante scegliere

la variante Ok restituisce x/y

Err è un tipo stringa quindi stampo la stringa



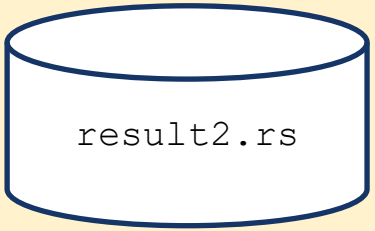
parse è una funzione che cerca di convertire una stringa a numero intero a 32 bit

11:55

parse ritorna un result e quindi può essere usato dalla clausola match

```
fn string_to_int(s: &str) -> Result<i32, &'static str> {
    match s.parse::<i32>() {
        Ok(n) => Ok(n),
        Err(_) => Err("Impossibile convertire la stringa in un numero intero."),
    }
}

fn main() {
    match string_to_int("mamma") {
        Ok(num) => println!("Il numero convertito è: {}", num),
        Err(message) => println!("{}", message),
    }
}
```

usando l'underscore IGNORO il tipo che viene ritornato dal parse e creo un tipo che desidero, come ad esempio una stringa



chiedi cosa rappresenta "num" e "n" in questo caso

Per saperne di più

- Rust Basics: Structs, Methods, and Traits
  - https://medium.com/better-programming/rust-basics-structs-methods-and-traits-bb4839cd57bd
- Mastering Enums in Rust: Best Practices and Examples
  - https://sterlingcobb.medium.com/mastering-enums-in-rust-best-practices-and-examples-a0bd76ea8cf
- Enums and Pattern Matching in Rust
  - https://medium.com/better-programming/rust-enums-and-pattern-matching-177b03a4152
- Destructuring
  - https://aminb.gitbooks.io/rust-for-c/content/destructuring/index.html
  - https://aminb.gitbooks.io/rust-for-c/content/destructuring_2/index.html

Glossario

- Struttura (struct): tipo di dato personalizzato che permette di raggruppare dati correlati
- Tuple struct: tipo di struct con campi senza nome
- Unit struct: tipo di struct senza campi
- Istanza di una struct: allocazione di una nuova variabile avente come tipo quello dichiarato dalla struct
- Incapsulamento: controllo della visibilità di moduli e dei loro elementi (struct, enum, funzioni, ecc.) usando la parola chiave pub per renderli pubblici e accessibili all’esterno
- Visibilità: proprietà che permette di nascondere i dettagli implementativi interni e esporre un'interfaccia pubblica
- Modificatore di visibilità: si utilizza la parola chiave pub per rendere visibile un modulo e i loro elementi

© G. Malnati, G. Liaci, 2021-24

Glossario

- Tipo algebrico: tipo di dato composto formando combinando altri tipi
- Tipo Somma: insieme di possibili valori che un tipo può assumere è la somma disgiunta (o unione) dei diversi tipi possibili
- Tipo Prodotto: insieme di possibili valori che un tipo può assumere è il prodotto cartesiano dei possibili valori di diversi tipi

© G. Malnati, G. Liaci, 2021-24

Glossario

- Metodo: funzione associata ad un tipo specifico, come una struct o un enum
- Ricevitore: l'istanza dell'oggetto su cui viene chiamato il metodo
- Funzione associata (o metodo statico): Metodo che non ha self come primo parametro
- Costruttore: Metodo utilizzato per inizializzare una nuova istanza
- Distruttore: Funzione (drop) del tratto Drop chiamato nel momento in cui una variabile esce dal suo scope

© G. Malnati, G. Liaci, 2021-24

Glossario

- Enumerazione: tipo che può assumere un numero finito di valori possibili che sono definiti esplicitamente. Le enumerazioni sono utilizzate per rappresentare un insieme di stati possibili o opzioni distinte
- Varianti: possibili alternative che una enumerazione può assumere
- Tag: informazione (implicita o esplicita) che indica quale variante dell'enumerazione è attualmente memorizzata in una istanza di quel tipo
- Padding: inserimento in memoria di byte aggiuntivi (e non significativi) tra i campi di una struttura dati con l'obiettivo di allineare correttamente i dati in memoria in base ai requisiti dell'architettura del processore.

© G. Malnati, G. Liaci, 2021-24